

GUIA CONCEITUAL E EXECUTIVO

Sistema de Gerenciamento de Aviação Comercial (Java Desktop)

Contexto: Oficina de Integração de Software / Faculdade	Metodologia: Personal Scrum (Solo Dev)
Ambiente Acadêmico: Eclipse IDE (Windows) + Maven + phpMyAdmin	Status do Projeto: Alinhamento & Planejamento de MVP

1. Visão Geral e Objetivo do Projeto

Este documento serve como mapa estratégico e técnico para o desenvolvimento independente de uma aplicação desktop em **Java**, projetada para funcionar como um sistema gerenciador de banco de dados para aviação comercial. O projeto aproveita uma modelagem relacional pré-existente (composta por tabelas de passageiros, aeronaves, companhias aéreas, voos e reservas), implementando uma camada robusta de software que provê operações de inserção (*setters*) e consulta (*getters*).

O grande desafio desta iniciativa é o ingresso tardio na oficina de software. Para alcançar o progresso da equipe que desenvolve em outra stack (Python), este plano adota práticas ágeis adaptadas para um único desenvolvedor (**Personal Scrum**), garantindo entregas incrementais focadas em um Produto Mínimo Viável (MVP) validável pelo professor.

2. Ambiente de Trabalho e Infraestrutura Acadêmica

O desenvolvimento e as apresentações oficiais ocorrerão obrigatoriamente nos computadores da faculdade. Portanto, o ambiente deve ser rigorosamente replicado e compatibilizado.

2.1. Especificações Técnicas do Ambiente Alvo

- **Sistema Operacional do Laboratório:** Microsoft Windows.
- **IDE Oficial:** Eclipse IDE (configurado com suporte nativo a projetos Maven).
- **Gerenciador de Dependências:** Apache Maven (utilizando o arquivo `pom.xml` para gerenciar as bibliotecas externas sem a necessidade de inclusão manual de arquivos `.jar`).
- **Banco de Dados e Administração:** Servidor MariaDB/MySQL gerenciado através da interface web **phpMyAdmin** (disponibilizado via pacote XAMPP ou instalação equivalente do laboratório).

Atenção com a Portabilidade (Linux vs. Windows): Como o desenvolvimento doméstico utiliza Linux (XAMPP/MariaDB 10.4.22) e a faculdade adota Windows, o código Java deve utilizar caminhos relativos e propriedades de conexão puramente via rede local (`localhost` / `127.0.0.1`). A string de conexão JDBC permanecerá idêntica em ambos os sistemas, garantindo portabilidade imediata.

3. Estado Atual do Desenvolvimento (O que já foi feito)

O projeto já possui sua fundação estrutural e de dados completamente estabelecida e testada localmente:

1. **Instalação do SGBD e Ferramental:** O ambiente de banco de dados (XAMPP/MariaDB) foi instalado e configurado com sucesso.
2. **Criação e Carga do Banco de Dados:** O arquivo de dump SQL (`sistema_aviao.sql`) foi importado com êxito na interface do phpMyAdmin, criando a estrutura relacional completa e populando o banco com dados iniciais (10 registros por tabela para testes).
3. **Mapeamento das Entidades:** A arquitetura de tabelas foi analisada, identificando as entidades independentes (Passageiro, Aeroporto, Cia_Aerea) e as dependentes que possuem restrições de chaves estrangeiras (Aeronave_Cia, Voo, Reserva).

4. Escopo do MVP (Mínimo Produto Viável)

Para a apresentação de validação para o professor, o foco será a entrega de uma interface textual (Menu via Console) ou gráfica simplificada focada nas operações de consulta (get) e inserção (set) de dados, priorizando a tabela de **Aeronaves**, mas estruturada para suportar as demais entidades.

4.1. Estrutura Proposta para o Menu do Sistema (Interface do Usuário)

Ao iniciar a aplicação no Eclipse, o usuário visualizará um menu interativo e navegável:

```
=====
                SISTEMA DE GERENCIAMENTO DE AVIAÇÃO
=====
1. GERENCIAR AERONAVES (Consultar / Cadastrar)
2. GERENCIAR COMPANHIAS AÉREAS
3. GERENCIAR PASSAGEIROS
4. CONSULTAR VOOS E DISPONIBILIDADE
5. EFETUAR RESERVA DE PASSAGEIRO
0. SAIR DO SISTEMA
-----
Escolha uma opção: _
```

4.2. Fluxo da Operação da Entidade Foco (Aeronaves)

- **Consulta (Get):** O sistema listará na tela o modelo, fabricante, capacidade e status ativo de todas as aeronaves cadastradas, lendo diretamente do MariaDB. Permitirá também a busca filtrada pelo ID da aeronave.
- **Inserção (Set):** O formulário solicitará os campos necessários (Modelo, Capacidade, Envergadura, Fabricante, Status Ativo), efetuará a validação dos tipos de dados e executará o `INSERT INTO aeronave` de forma segura.

5. Roadmap de Desenvolvimento (Personal Scrum)

Como o projeto é conduzido por um único desenvolvedor, as Sprints são compactas e focadas em componentes arquiteturais específicos para mitigar riscos de integração.

Sprint	Meta Principal	Entregáveis / Atividades Técnicas
Sprint 1	Infraestrutura Java e Conexão	Configuração do <code>pom.xml</code> com a dependência <code>mysql-connector-j</code> . Criação da classe utilitária <code>ConnectionFactory</code> utilizando JDBC para gerenciar conexões com o banco local.
Sprint 2	Camada Model e DAOs Base	Criação dos POJOs (classes de modelo) e das classes DAO (Data Access Object) para as entidades independentes: <code>Passageiro</code> , <code>Aeroporto</code> e <code>CiaAerea</code> . Testes de CRUD via console.
Sprint 3	Lógica Relacional e Regras	Implementação das entidades complexas (<code>Aeronave</code> , <code>Voo</code> , <code>Reserva</code>) que exigem amarração de chaves estrangeiras. Tratamento de exceções SQL (Integridade Referencial).
Sprint 4	Interface do MVP e Polimento	Construção da interface de Menu (Console ou Swing), integração total com os métodos DAO e empacotamento do projeto prontos para importação no Eclipse do laboratório.

6. Visão Futura: Definição de Pronto (Definition of Done - DoD)

Para garantir que o projeto não possua "pontas soltas" e atenda aos critérios rigorosos de avaliação acadêmica e de engenharia de software, nenhuma funcionalidade será considerada concluída ou pronta para demonstração se não cumprir todos os critérios da **Definição de Pronto (DoD)** abaixo:

Checklist da Definição de Pronto (DoD)

1. **■ Persistência Real:** Toda inserção feita pela interface deve refletir imediatamente no banco de dados visível via phpMyAdmin (sem dados mockados em memória).
2. **■ Tratamento de Exceções (Fail-Safe):** Tentativas de inserir chaves estrangeiras inexistentes (ex: criar um voo em um aeroporto não cadastrado) devem ser capturadas e exibidas como alertas amigáveis ao usuário, sem derrubar a aplicação (crash do sistema).
3. **■ Gerenciamento Construtivo de Recursos:** Toda conexão aberta com o banco através do driver JDBC deve ser explicitamente encerrada (uso de `try-with-resources` ou blocos `finally`) para evitar vazamento de memória e travamento do servidor MariaDB.
4. **■ Compilação Limpa no Maven:** O projeto deve executar o comando `mvn clean compile` no Eclipse sem apresentar nenhum erro de ciclo de vida ou dependência ausente.
5. **■ Portabilidade de Ambiente Windows:** O código-fonte deve rodar imediatamente ao ser importado no Eclipse sob o sistema operacional Windows do laboratório, exigindo apenas a alteração das credenciais de banco se necessário.